MATH 128 - HW5 - Jie Wen

Problem 1: Chapter 1

(a)

The Taylor series for the exponential function e^x around $x = 0$ is given by:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

Substituting $x = -\frac{1}{n^2}$ into this expansion:

$$e^{-\frac{1}{n^2}} = 1 - \frac{1}{n^2} + \frac{1}{2! \cdot n^4} - \frac{1}{3! \cdot n^6} + \cdots = 1 - \frac{1}{n^2} + R_n$$

where the remainder term R_n is:

$$R_n = \frac{1}{2! \cdot n^4} - \frac{1}{3! \cdot n^6} + \frac{1}{4! \cdot n^8} - \cdots$$

For large n, the series is absolutely convergent, and the remainder is bounded by the absolute value of the first omitted term.

For $n \geq 1$, the absolute value of the remainder can be bounded:

$$|R_n| \leq \frac{1}{2! \cdot n^4} + \frac{1}{3! \cdot n^6} + \frac{1}{4! \cdot n^8} + \cdots$$

Since $n^{-4} > n^{-6} > n^{-8} > \cdots$ for $n \geq 1$, each term in the sum is less than or equal to $\frac{1}{n^4}$ multiplied by a constant. Factoring out $\frac{1}{n^4}$ gives:

$$|R_n| \leq \frac{1}{n^4}\left(\frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \cdots\right)$$

The series inside the parentheses converges to a constant (specifically, $e - 2 \approx 0.71828$). Therefore, there exists a constant $C_1 > 0$ such that:

$$|R_n| \leq \frac{C_1}{n^4}$$

Since $\frac{1}{n^4} = O(n^{-4})$ and $O(n^{-4})$ implies $O(n^{-2})$ because n^{-4} decays faster than n^{-2}, it follows that $R_n = O(n^{-2})$.
Therefore:

$$e^{-\frac{1}{n^2}} - 1 = O(n^{-2})$$

(b)

Let $f(x) = x^2 - 4x^3 + 1$

- At $x = 0$: $f(0) = 0^2 - 4(0)^3 + 1 = 1$
- At $x = 1$: $f(1) = 1^2 - 4(1)^3 + 1 = 1 - 4 + 1 = -2$

Since $f(x)$ is a polynomial, it is continuous on $[0, 1]$:

$$f(0) = 1 > 0 \quad \text{and} \quad f(1) = -2 < 0$$

By the **Intermediate Value Theorem**, there exists at least one $c \in (0, 1)$ such that

$$f(c) = 0$$

(c)

Differentiate the function:

$$f'(x) = \frac{1}{(1+x)^2}$$

Since

$$f'(x) > 0 \quad \text{for all } x \in [0, 4],$$

the function $f(x)$ is **strictly increasing** on the interval $[0, 4]$.

- **Minimum** occurs at the left endpoint:

$$f(0) = \frac{0}{1+0} = 0$$

- **Maximum** occurs at the right endpoint:

$$f(4) = \frac{4}{1+4} = \frac{4}{5}$$

(d)

The third Taylor polynomial is:

$$P_3(x) = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3$$

1. $\sqrt{0.5} \approx P_3(-0.5)$

$$P_3(-0.5) = 1 + \frac{1}{2}(-0.5) - \frac{1}{8}(0.25) + \frac{1}{16}(-0.125) = 0.7109375$$

Actual value:

$$\sqrt{0.5} \approx 0.70710678$$

Actual error:

$$0.70710678 - 0.7109375 = -0.00383072$$

2. $\sqrt{0.75} \approx P_3(-0.25)$

$$P_3(-0.25) = 1 + \frac{1}{2}(-0.25) - \frac{1}{8}(0.0625) + \frac{1}{16}(-0.015625) = 0.8662109375$$

Actual value:

$$\sqrt{0.75} \approx 0.8660254$$

Actual error:

$$0.8660254 - 0.8662109375 = -0.0001855375$$

3. $\sqrt{1.25} \approx P_3(0.25)$

$$P_3(0.25) = 1 + \frac{1}{2}(0.25) - \frac{1}{8}(0.0625) + \frac{1}{16}(0.015625) = 1.1181640625$$

Actual value:

$$\sqrt{1.25} \approx 1.11803399$$

Actual error:

$$1.11803399 - 1.1181640625 = -0.0001300725$$

4. $\sqrt{1.5} \approx P_3(0.5)$

$$P_3(0.5) = 1 + \frac{1}{2}(0.5) - \frac{1}{8}(0.25) + \frac{1}{16}(0.125) = 1.2265625$$

Actual value:

$$\sqrt{1.5} \approx 1.22474487$$

Actual error:

$$1.22474487 - 1.2265625 = -0.00181763$$

(e)

The n-th Taylor polynomial:

$$P_n(x) = 1 + x + x^2 + \cdots + x^n = \sum_{k=0}^{n} x^k$$

The error in the Taylor approximation is the remainder:

$$R_n(x) = f(x) - P_n(x) = x^{n+1} + x^{n+2} + \cdots = \frac{x^{n+1}}{1-x}$$

We want:

$$|R_n(x)| = \left| \frac{x^{n+1}}{1-x} \right| \leq 10^{-6} \quad \text{for all } x \in [0, 0.5]$$

On the interval $[0, 0.5]$, the denominator $1 - x$ is minimized at $x = 0.5$, and the numerator x^{n+1} is maximized at $x = 0.5$. Thus, the maximum error occurs at $x = 0.5$:

$$|R_n(x)| \leq \frac{(0.5)^{n+1}}{0.5} = (0.5)^n$$

Now require:

$$(0.5)^n \leq 10^{-6}$$

Taking logarithms:

$$n \ln(0.5) \leq \ln(10^{-6}) = -6 \ln(10)$$

$$n \geq \frac{6 \ln(10)}{\ln(2)} \approx 19.93$$

Since n must be an integer:

$$n = 20$$

(f)

(b) $p = \pi, p^* = 3.1416$

- $\pi \approx 3.141592653589793$
- Absolute Error:

$$E_{\text{abs}} = |3.141592653589793 - 3.1416| \approx 7.34641 \times 10^{-6}$$

- Relative Error:

$$E_{\text{rel}} = \frac{7.34641 \times 10^{-6}}{3.141592653589793} \approx 2.338 \times 10^{-6}$$

(c) $p = e, p^* = 2.718$

- $e \approx 2.718281828459045$
- Absolute Error:

$$E_{\text{abs}} = |2.718281828459045 - 2.718| \approx 2.81828 \times 10^{-4}$$

- Relative Error:

$$E_{\text{rel}} = \frac{2.81828 \times 10^{-4}}{2.718281828459045} \approx 1.0368 \times 10^{-4}$$

(g)

$$2^1 + 2^3 + 2^{10} = 2034, \quad 2034 - 2023 = 11$$

$$f = 0.5 + 0.0625 + 0.0078125 + 0.00390625 = 0.57421875$$

$$x = (-1) \times 1.57421875 \times 2^{11} = -3224$$

The gap between consecutive numbers in this range is $2^{E-52} = 2^{11-52} = 2^{-41}$

- **Next largest**: $-3224 + 2^{-41}$
- **Next smallest**: $-3224 - 2^{-41}$

Problem 2: Bisection method

(a) Debugging the Code

The main issues with the code are:

1. **Incorrect interval update logic**: The condition `if f(a)*f(b)>0` is wrong. Bisection requires that we check which subinterval contains the root, not whether the function has the same sign at both endpoints.
2. **Missing initial bracket verification**: The code doesn't verify that `f(a)` and `f(b)` have opposite signs initially, which is required for bisection to work.
3. **Incorrect midpoint evaluation**: The current logic would fail if the initial bracket doesn't contain a root.

Below is the corrected code:

```
function [p,nit] = bisection_corrected(f,a,b,tol)
    if nargin < 4
        tol = 10^(-6);
        if nargin < 2
            a = 0;
            b = 1;
            if nargin < 1
                f = @(x)(x^2-0.5);
            end
        end
    end

    % Verify initial bracket contains a root
    if f(a)*f(b) >= 0
        error('Bisection method requires f(a) and f(b) have opposite signs');
    end

    for k = 1:10000
        p = (a+b)/2;
        nit = k;

        % Original stopping criterion
        if abs(p-a) < tol || f(p) == 0
            break;
        end

        % Correct interval update
        if f(a)*f(p) < 0
            b = p;  % Root is in [a,p]
        else
            a = p;  % Root is in [p,b]
        end
    end
end
```

(b) Alternative Stopping Criterion

```
function [p,nit] = bisection_function_tol(f,a,b,tol)
    if nargin < 4
        tol = 10^(-6);
        if nargin < 2
            a = 0;
            b = 1;
            if nargin < 1
                f = @(x)(x^2-0.5);
            end
        end
    end

    % Verify initial bracket contains a root
    if f(a)*f(b) >= 0
        error('Bisection method requires f(a) and f(b) have opposite signs');
    end

    for k = 1:10000
        p = (a+b)/2;
        nit = k;

        % New stopping criterion: stop when |f(p)| < tol
        if abs(f(p)) < tol
            break;
        end

        % Correct interval update
        if f(a)*f(p) < 0
            b = p;
```

```
        else
            a = p;
        end
    end
end
```

Comparison - Key differences:

1. **Interval-based stopping**: Guarantees the root is within a specified distance, but might continue iterating even when very close to the actual root.
2. **Function value-based stopping**: Stops when the function value is sufficiently small, which might happen before or after the interval-based criterion depending on the function's slope near the root.
3. **Robustness**: Interval-based is generally more reliable for ill-conditioned functions where small changes in x cause large changes in $f(x)$.

Problem 3: Fixed point iteration

MATLAB codes:

```
% Define the function and its derivative
f = @(x) x.^3 - 2*x + 1;
f_prime = @(x) 3*x.^2 - 2;

% Initial guess
p0 = 0.5;
fprintf('Initial guess: p0 = %.4f, f(p0) = %.6f\n\n', p0, f(p0));

% Define the fixed-point functions
g_a = @(x) (x^3 + 1)/2;
g_b = @(x) -2/x - 1/x^2;
g_c = @(x) sqrt(2 - 1/x);
g_d = @(x) -sqrt(1 - 2*x);

% Define derivatives for convergence analysis
g_prime_a = @(x) (3*x^2)/2;
g_prime_b = @(x) 2/x^2 + 2/x^3;
g_prime_c = @(x) (1/(2*x^2)) / sqrt(2 - 1/x);
g_prime_d = @(x) 1/sqrt(1 - 2*x);

% Method (a): x = (x^3 + 1)/2
fprintf('=== METHOD (a): x = (x^3 + 1)/2 ===\n');
p = p0;
iterations_a = zeros(1, 5);
iterations_a(1) = p;
errors_a = zeros(1, 4);

fprintf('k\tp_k\t\tf(p_k)\t\t|p_k - p_{k-1}|\t|g''(p_k)|\n');
fprintf('0\t%.6f\t%.6f\t-\t\t%.6f\n', p, f(p), abs(g_prime_a(p)));

for k = 1:20
    p_old = p;
    p = g_a(p);
    iterations_a(k+1) = p;
    errors_a(k) = abs(p - p_old);

    fprintf('%d\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
            k, p, f(p), errors_a(k), abs(g_prime_a(p)));
end

% Check convergence condition
fprintf('\nConvergence analysis:\n');
fprintf('|g''(p_4)| = %.6f %s 1\n', abs(g_prime_a(p)), ...
        char('<' * (abs(g_prime_a(p)) < 1) + '>' * (abs(g_prime_a(p)) >= 1)));

% Method (b): x = -2/x - 1/x^2
fprintf('\n\n=== METHOD (b): x = -2/x - 1/x^2 ===\n');
p = p0;
iterations_b = zeros(1, 5);
iterations_b(1) = p;
errors_b = zeros(1, 4);

fprintf('k\tp_k\t\tf(p_k)\t\t|p_k - p_{k-1}|\t|g''(p_k)|\n');
fprintf('0\t%.6f\t%.6f\t-\t\t%.6f\n', p, f(p), abs(g_prime_b(p)));

for k = 1:20
    p_old = p;
    try
        p = g_b(p);
        iterations_b(k+1) = p;
        errors_b(k) = abs(p - p_old);
```

```
        fprintf('%d\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
                k, p, f(p), errors_b(k), abs(g_prime_b(p)));
    catch ME
        fprintf('%d\tERROR: %s\n', k, ME.message);
        break;
    end
end

% Method (c): x = sqrt(2 - 1/x)
fprintf('\n\n=== METHOD (c): x = sqrt(2 - 1/x) ===\n');
p = p0;
iterations_c = zeros(1, 5);
iterations_c(1) = p;
errors_c = zeros(1, 4);

fprintf('k\tp_k\t\tf(p_k)\t\t|p_k - p_{k-1}|\t|g''(p_k)|\n');
fprintf('0\t%.6f\t%.6f\t-\t\t%.6f\n', p, f(p), abs(g_prime_c(p)));

for k = 1:20
    p_old = p;
    try
        p = g_c(p);
        iterations_c(k+1) = p;
        errors_c(k) = abs(p - p_old);

        fprintf('%d\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
                k, p, f(p), errors_c(k), abs(g_prime_c(p)));
    catch ME
        fprintf('%d\tERROR: %s\n', k, ME.message);
        break;
    end
end

% Method (d): x = -sqrt(1 - 2x)
fprintf('\n\n=== METHOD (d): x = -sqrt(1 - 2x) ===\n');
p = p0;
iterations_d = zeros(1, 5);
iterations_d(1) = p;
errors_d = zeros(1, 4);

fprintf('k\tp_k\t\tf(p_k)\t\t|p_k - p_{k-1}|\t|g''(p_k)|\n');
fprintf('0\t%.6f\t%.6f\t-\t\t%.6f\n', p, f(p), abs(g_prime_d(p)));

for k = 1:20
    p_old = p;
    try
        p = g_d(p);
        iterations_d(k+1) = p;
        errors_d(k) = abs(p - p_old);

        fprintf('%d\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
                k, p, f(p), errors_d(k), abs(g_prime_d(p)));
    catch ME
        fprintf('%d\tERROR: %s\n', k, ME.message);
        break;
    end
end

% Summary and comparison
fprintf('\n\n=== SUMMARY ===\n');

% Visualization
figure('Position', [100, 100, 1200, 800]);

% Plot 1: Function and roots
subplot(2, 3, 1);
x = linspace(-2, 2, 1000);
plot(x, f(x), 'b-', 'LineWidth', 2); hold on;
plot(x, zeros(size(x)), 'k--', 'LineWidth', 1);
xlabel('x'); ylabel('f(x)');
title('f(x) = x^3 - 2x + 1');
grid on;

% Plot iterations for each method
colors = ['r', 'g', 'm', 'c'];
methods = {'(a)', '(b)', '(c)', '(d)'};
iter_data = {iterations_a, iterations_b, iterations_c, iterations_d};

for i = 1:4
    subplot(2, 3, i+1);
    valid_iters = iter_data{i}(iter_data{i} ~= 0);
    if ~isempty(valid_iters)
        plot(0:length(valid_iters)-1, valid_iters, 'o-', ...
             'Color', colors(i), 'LineWidth', 2, 'MarkerFaceColor', colors(i));
        xlabel('Iteration'); ylabel('p_k');
```

```
        title(sprintf('Method %s Iterations', methods{i}));
        grid on;
    else
        text(0.5, 0.5, 'Method Failed', 'HorizontalAlignment', 'center', ...
             'Units', 'normalized', 'FontSize', 12);
        title(sprintf('Method %s', methods{i}));
    end
end

% Plot error convergence
subplot(2, 3, 6);
for i = 1:4
    switch i
        case 1
            errors = errors_a;
        case 2
            errors = errors_b;
        case 3
            errors = errors_c;
        case 4
            errors = errors_d;
    end
    valid_errors = errors(errors ~= 0 & ~isnan(errors) & isfinite(errors));
    if ~isempty(valid_errors)
        semilogy(1:length(valid_errors), valid_errors, 'o-', ...
                'Color', colors(i), 'LineWidth', 2, 'MarkerFaceColor', colors(i));
        hold on;
    end
end
xlabel('Iteration'); ylabel('|p_k - p_{k-1}|');
title('Error Convergence');
legend('Method (a)', 'Method (b)', 'Method (c)', 'Method (d)', 'Location', 'best');
grid on;

% Find actual roots for comparison
actual_roots = roots([1, 0, -2, 1]);
fprintf('\nActual roots of f(x) = 0:\n');
for i = 1:length(actual_roots)
    if abs(imag(actual_roots(i))) < 1e-10
        fprintf('Root %d: %.6f\n', i, real(actual_roots(i)));
    end
end
```

Output:

```
Initial guess: p0 = 0.5000, f(p0) = 0.125000

=== METHOD (a): x = (x^3 + 1)/2 ===
k   p_k     f(p_k)      |p_k - p_{k-1}| |g'(p_k)|
0   0.500000    0.125000    -       0.375000
1   0.562500    0.052979    0.062500    0.474609
2   0.588989    0.026347    0.026489    0.520363
3   0.602163    0.014019    0.013173    0.543900
4   0.609172    0.007714    0.007009    0.556636
5   0.613029    0.004321    0.003857    0.563707
6   0.615190    0.002444    0.002161    0.567687
7   0.616412    0.001390    0.001222    0.569945
8   0.617107    0.000793    0.000695    0.571232
9   0.617504    0.000453    0.000397    0.571966
10  0.617730    0.000259    0.000227    0.572386
11  0.617860    0.000149    0.000130    0.572627
12  0.617934    0.000085    0.000074    0.572764
13  0.617977    0.000049    0.000043    0.572843
14  0.618001    0.000028    0.000024    0.572888
15  0.618015    0.000016    0.000014    0.572914
16  0.618023    0.000009    0.000008    0.572929
17  0.618028    0.000005    0.000005    0.572938
18  0.618030    0.000003    0.000003    0.572942
19  0.618032    0.000002    0.000002    0.572945
20  0.618033    0.000001    0.000001    0.572947

Convergence analysis:
|g'(p_4)| = 0.572947 < 1


=== METHOD (b): x = -2/x - 1/x^2 ===
k   p_k     f(p_k)      |p_k - p_{k-1}| |g'(p_k)|
0   0.500000    0.125000    -       24.000000
1   -8.000000   -495.000000 8.500000    0.027344
2   0.234375    0.544125    8.234375    191.753481
3   -26.737778  -19060.596014   26.972153   0.002693
4   0.073402    0.853592    26.811180   5428.414688
5   -212.851227 -9642935.446715 212.924629  0.000044
```

```
6   0.009374    0.981252    212.860602  2450669.911283
7   -11593.164553   -1558138269299.659668   11593.173927    0.000000
8   0.000173    0.999655    11593.164726    389652184547.766541
9   -33614858.232597    -37983401307419422228480.000000 33614858.232769 0.000000
10  0.000000    1.000000    33614858.232597 9495851315568761110528.000000
11  -282489715517971.625000 -225428034168078412762888985551918608611540992.000000    282489715517971.625000  0.000000
12  0.000000    1.000000    282489715517971.625000  563570085420202964371442461927504650384179 2.000000
13  -19950109843356486749407674368.000000
-7940281029513594926933989657896251546946354634574430399874516966115177386373061541888.000000
19950109843356486749407674368.000000    0.000000
14  0.000000    1.000000    19950109843356486749407674368.000000
19850702573783987317334974144740628867365886586436075999686292415287943465932653854 72.000000
15  -99501720690497349693607343481462862822682976176127868928.000000
-98512598168208577087816001179887350832067700812613280355673100470860982842907311800237832233042622011016589655538349300568493106647811386574848618813320510451726732492 8.000000  99501720690497349693607343481462862822682976176127868928.000000 0.000000
16  0.000000    1.000000    99501720690497349693607343481462862822682976176127868928.000000
24628149542052149509378972928798829729120440127311963635545548786618908883993016903873532305739943315986295135331238766560817880758066461391322628119959012938575826124 8.000000
17  -247514810509243749876522203559622190008023111880469934816672677258902500813126663617791280060868515712224093798 4.000000    -Inf 24751481050924374987652220355962219000802311188046993481667267725890250081312666361779128006086851571222409379 84.000000
0.000000
18  0.000000    1.000000
24751481050924374987652220355962219000802311188046993481667267725890250081312666361779128006086851571222409379 84.000000 Inf
19
-153158953553567095104005481926615920316532844638989484836066589419097028222531492421736349866379171942053057908527515008606538711955753640330275550797402724196277920510615015662820908745067159015275299767974465588246515548160.000000   -Inf
153158953553567095104005481926615920316532844638989484836066589419097028222531492421736349866379171942053057908527515008605387119557536403302755507974027241962779205106150156628209087450671590152752997679744655882465155481 60.000000    0.000000
20  0.000000    1.000000
153158953553567095104005481926615920316532844638989484836066589419097028222531492421736349866379171942053057908527515008605387119557536403302755507974027241962779205106150156628209087450671590152752997679744655882465155481 60.000000    Inf

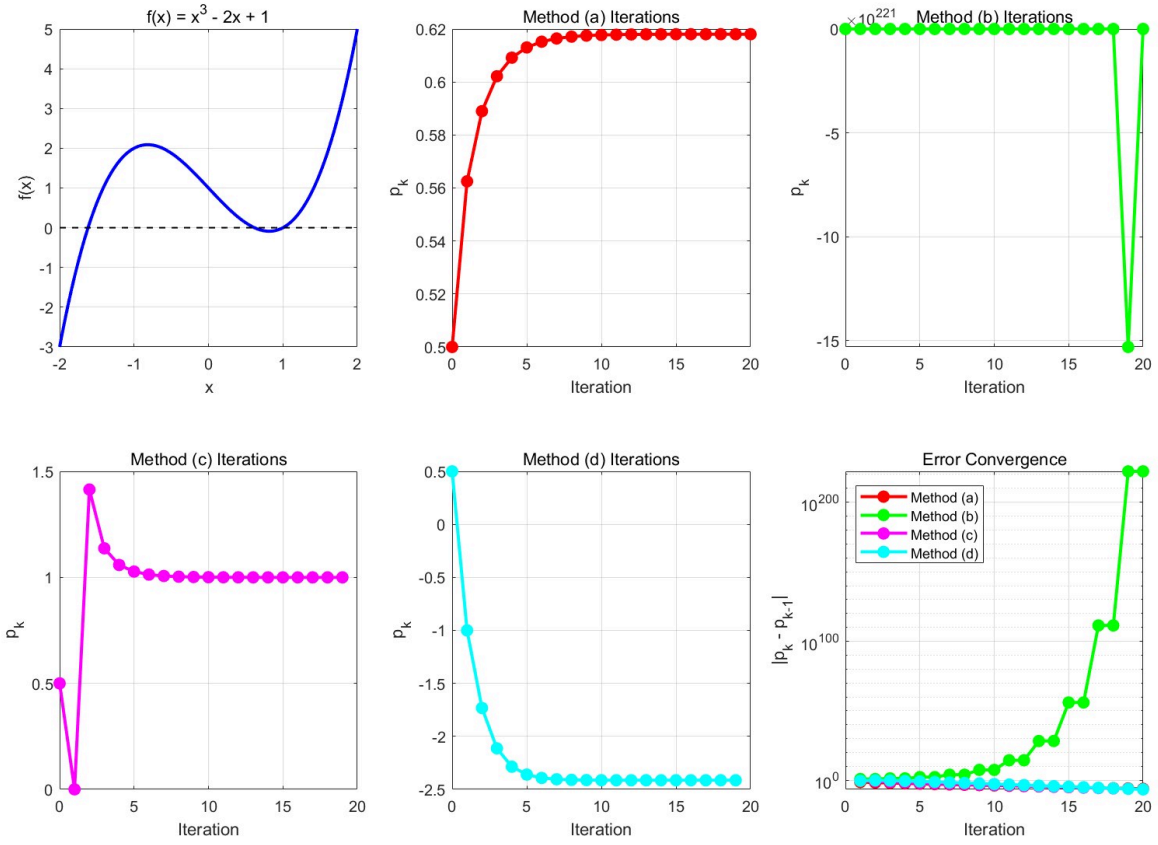

=== METHOD (c): x = sqrt(2 - 1/x) ===
k   p_k     f(p_k)      |p_k - p_{k-1}| |g'(p_k)|
0   0.500000    0.125000    -       Inf
1   0.000000    1.000000    0.500000    NaN
2   0.000000    1.000000    Inf 0.000000
3   1.414214    1.000000    Inf 0.219866
4   1.137055    0.195981    0.277159    0.365338
5   1.058553    0.069039    0.078501    0.434364
6   1.027285    0.029539    0.031268    0.467623
7   1.013193    0.013718    0.014092    0.483923
8   1.006490    0.006616    0.006703    0.491989
9   1.003219    0.003250    0.003271    0.496002
10  1.001603    0.001611    0.001616    0.498003
11  1.000800    0.000802    0.000803    0.499002
12  1.000400    0.000400    0.000400    0.499501
13  1.000200    0.000200    0.000200    0.499751
14  1.000100    0.000100    0.000100    0.499875
15  1.000050    0.000050    0.000050    0.499938
16  1.000025    0.000025    0.000025    0.499969
17  1.000012    0.000012    0.000012    0.499984
18  1.000006    0.000006    0.000006    0.499992
19  1.000003    0.000003    0.000003    0.499996
20  1.000002    0.000002    0.000002    0.499998


=== METHOD (d): x = -sqrt(1 - 2x) ===
k   p_k     f(p_k)      |p_k - p_{k-1}| |g'(p_k)|
0   0.500000    0.125000    -       Inf
1   -0.000000   1.000000    0.500000    1.000000
2   -1.000000   2.000000    1.000000    0.577350
3   -1.732051   -0.732051   0.732051    0.473296
4   -2.112842   -4.206258   0.380791    0.437450
5   -2.285976   -6.373835   0.173133    0.423639
6   -2.360498   -7.431584   0.074523    0.418085
7   -2.391860   -7.900103   0.031362    0.415811
8   -2.404937   -8.099609   0.013076    0.414874
9   -2.410368   -8.183197   0.005431    0.414487
10  -2.412620   -8.217984   0.002252    0.414327
11  -2.413553   -8.232422   0.000933    0.414260
12  -2.413940   -8.238407   0.000387    0.414233
13  -2.414100   -8.240887   0.000160    0.414222
14  -2.414167   -8.241914   0.000066    0.414217
15  -2.414194   -8.242340   0.000027    0.414215
16  -2.414206   -8.242516   0.000011    0.414214
17  -2.414210   -8.242589   0.000005    0.414214
18  -2.414212   -8.242619   0.000002    0.414214
19  -2.414213   -8.242632   0.000001    0.414214
20  -2.414213   -8.242637   0.000000    0.414214


=== SUMMARY ===
警告: 复数 X 和/或 Y 参数的虚部已忽略。
```

```
> 位置: qlast (第 147 行)

Actual roots of f(x) = 0:
Root 1: -1.618034
Root 2: 1.000000
Root 3: 0.618034
```



Actual roots of $f(x) = x^3 - 2x + 1$:

$$\text{roots} \approx -1.618034; 1.000000; 0.618034.$$

- **Method (a)**: converges to ≈ 0.618034.
  - *Evidence:* iterates march steadily toward 0.6180339; at the limit $|g_a'(0.6180)| \approx 0.57295 < 1$.
  - *Conclusion:* **appropriate** — it is a contracting map near that root (slow but steady convergence).
- **Method (b)**: *unstable / chaotic* for this starting point.
  - *Evidence:* huge oscillations, entries equal to 0, extremely large magnitudes, repeated near-singular values (division by very small numbers). The derivative $g_b'(x)$ blows up near $x = 0$.
  - *Conclusion:* **not appropriate** (very sensitive to initial guess and has singularities at $x = 0$).
- **Method (c)**: converges to the root $x = 1.0$ eventually.
  - *Evidence:* after some problematic early iterates (your run shows $p_1 = 0$ then $p_2 = 0$, then it jumps to $\sqrt{2}$ and then settles toward 1.0); near $x = 1$, $|g_c'(1)| = 0.5 < 1$.
  - *Conclusion:* **usable but risky** — the formula involves $1/x$ and square root, so domain/branch issues and division-by-zero problems can occur (your run encountered $p = 0$ transiently). With a safer initial guess away from zero, this map behaves like a contraction to 1.
- **Method (d)**: converges, but **not** to a root of f.
  - *Evidence:* iterates approach ≈ -2.414213. That value is a fixed point of g_d because solving $x = -\sqrt{1 - 2x}$ gives $x^2 + 2x - 1 = 0 \Rightarrow x = -1 \pm \sqrt{2}$; the negative solution is $-1 - \sqrt{2} \approx -2.414213$. BUT that is **not** a root of f.
  - *Conclusion:* **inappropriate** for solving $f(x) = 0$ because the fixed points of this rearrangement do not coincide with the roots you want (it has its own fixed points unrelated to $f(x) = 0$).